BÖLÜM 3 / MODULE 3

Temel Esaslar: Aktörler, Deniz Seviyesi Hedefler & Paydaş Sözleşmeleri

Core Fundamentals: Actors, Sea-Level Goals & Contracts

Birincil ve ikincil aktörler, Alistair Cockburn'ün 3 adımlı adli testi (Etkileşim, Doğrulama, Durum Değişimi), Hedef İrtifaları (Bulut, Deniz Seviyesi, Balık), Kapsam sınırları ve In/Out listesi disiplini, 'Manage X' birleştir-sonra-böl kuralı ve kullanıcı dilinde isimlendirme.

Primary and secondary actors, Cockburn's three-kind step test (Interaction, Validation, State Change), Goal Altitude tiers (Cloud, Sea Level, Fish), Scope boundaries via the In/Out list, the 'Manage X' aggregate-then-split rule, and naming in user-centric language.

13

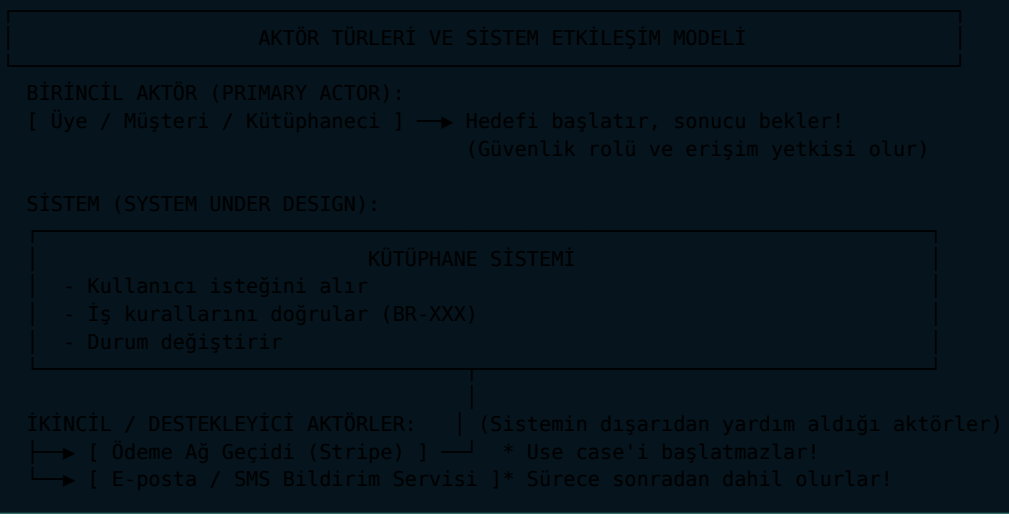
BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Aktörü Doğru Tanımlamak: Rol Odaklılık, Birincil Aktörler & İkincil Destekleyici Sistemler

Defining the Actor: Roles, Primary Actors & Supporting Systems

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir film çekimini düşünün: Birincil aktör filmin başrol oyuncusudur (Örn: James Bond). Olayları o başlatır, bombayı o imha eder, alkışı o toplar. İkincil aktör ise Bond'a telsizden bilgi veren veya arabasını getiren figüranlardır (Q veya araba kiralama şirketi). Eğer senaryoda başrolü arabayı kiralayan şirkete verirsen film saçma sapan bir hale gelir! Birincil aktör daima hedefin peşinden koşan kişidir!

🇬🇧 ENGLISH (ELI5)

In a spy thriller, the Primary Actor is James Bond. He initiates the mission, pursues the objective, and claims the victory. The Secondary Actors are the quartermaster (Q) who equips him with gadgets and the satellite network that relays coordinates. If you write the script from the perspective of the satellite instead of the secret agent, the narrative collapses into incoherence.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Aktör Tanımlamasının AIUP ve MAANG Mimarilerindeki Kritik Rolü: 1) İsim Değil Rol: Bir aktör asla 'Ahmet Bey' veya 'Muhasebe Departmanı' gibi kişisel veya departman bazlı tanımlanamaz; 'Üye' (Member), 'Kütüphaneci' (Librarian) gibi soyut roller olmalıdır. Çünkü bu roller doğrudan sistemin Güvenlik Modeline (RBAC - Role-Based Access Control) ve Spring Security rollerine dönüşür. AI kod üretirken `\@PreAuthorize("hasRole('MEMBER')")` annotasyonunu bu aktör tanımına göre koyar. 2) Birincil Aktör (Primary Actor): Senaryoyu tetikleyen ve use case'in hedefine ulaşmasından doğrudan fayda sağlayan aktördür. 3) İkincil / Destekleyici Aktörler (Secondary Actors): Senaryoyu başlatmazlar; sistem hedefe yürürken onların bir servisine ihtiyaç duyar (Ödeme Gateway'i, Fraud Kontrol Servisi vb.). Use case şablonunda 'Secondary Actors: Payment Gateway, Email Service' olarak açıkça listelenmelidir; aksi halde AI bu entegrasyonları mock'lamaz veya bağımlılıkları bağlamayı unutur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Actor semantics in enterprise systems: 1) Roles over Individuals: An actor must represent a systemic role ('Member', 'Underwriter'), never a persona or organizational hierarchy. This designation directly synthesizes the application's authorization boundary: an AI agent translates primary actor metadata into role-based security guards (`\@PreAuthorize`, JWT claims, and integration test credentials). 2) Primary Actor: Initiates the use case to satisfy a measurable goal. 3) Secondary / Supporting Actors: External systems or back-office actors enlisted by the system to complete sub-operations (e.g. Stripe, SendGrid, Fraud Detection APIs). Explicitly declaring secondary actors ensures downstream AI engines generate appropriate outbound ports and client interfaces within a Hexagonal architecture.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Aktör Rollerini Güvenlik ve Yetkilendirme Modeline Dönüştürme
package com.maang.aiup.actors;

public class ActorSecurityModel {
    // Şartnamede tanımlanan roller doğrudan Java 25 Enum/Record ile modellenir
    public enum SystemRole {
        MEMBER, LIBRARIAN, SYSTEM_ADMIN
    }

    public record SecurityContext(String userId, SystemRole role) {
        public void requireRole(SystemRole requiredRole) {
            if (this.role != requiredRole) {
                throw new SecurityException("Erişim reddedildi: Bu işlem için " + requiredRole + "
rolü zorunludur.");
            }
        }
    }

    // Birincil Aktör: Member
    public static void borrowBook(SecurityContext context, String bookId) {
        context.requireRole(SystemRole.MEMBER);
        System.out.println("Üye yetkisi doğrulandı, işlem başlıyor: " + bookId);
    }

    void main() {
        var memberCtx = new SecurityContext("user-42", SystemRole.MEMBER);
        borrowBook(memberCtx, "BOOK-1984");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Sistemin kendisini asla birincil aktör yapmayın! 'Zamanlanmış Görev (Cron)' veya 'Gece Batch İşlemi' gibi durumlarda aktör 'Sistem' değil, 'Zamanlayıcı Servisi' (Time / Scheduler) olmalıdır.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never designate the system itself as the primary actor. For automated background jobs, the primary actor is 'Time' or the 'Scheduler', which triggers the use case at a designated temporal threshold.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Aktör bir kişi veya departman değil, bir roldür; bu rol doğrudan güvenlik modelini besler.
- Birincil aktör hedefi başlatır ve faydayı alır; ikincil aktör ise sisteme destek verir.
- İkincil aktörleri açıkça listelemek, AI'ın dış servis entegrasyonlarını doğru modellemesini sağlar.

🎯 Key Architectural Takeaways

- An actor is a role, not a person or department; it directly feeds the security model.
- Primary actors pursue goals; secondary actors are external entities enlisted for support.
- Listing secondary actors forces AI to generate clean outbound API client boundaries.

14

BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Üç Adımlı Adli Test: Etkileşim, Doğrulama & Dahili Durum Değişimi

The Three-Step Test: Interaction, Validation & Internal State Change

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────┐
│              COCKBURN'ÜN 3 ADIMLI USE CASE ADLİ TESTİ │
└──────────────────────────────────────────────────────┘

Bir use case senaryosundaki HER ADIM mutlaka şu 3 türden biri olmak ZORUNDADIR:

TÜR 1: ETKİLEŞİM (INTERACTION)
└──► İki aktör arasında bilgi veya talep aktarımı.
     Örnek: "Üye ödünç almak istediği kitabı seçer ve talepte bulunur."

TÜR 2: DOĞRULAMA (VALIDATION)
└──► Sistemin bir paydaş çıkarını korumak için yaptığı kural kontrolü.
     Örnek: "Sistem üyenin gecikmiş borcu olmadığını doğrular (BR-006)."

TÜR 3: DURUM DEĞİŞİMİ (STATE CHANGE)
└──► Sistemin kalıcı belleğinde meydana gelen iş değişikliği.
     Örnek: "Sistem ödünç kaydını 28 gün vadeli olarak veritabanına işler."

⚠ ALARM: Eğer bir senaryoda HİÇ doğrulama ve durum değişimi yoksa,
         o bir use case değil; gizli bir 'Ekran Gezintisi'dir (UI Walkthrough)!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir dükkâna gidip alışveriş yapmayı düşünün: 1) Etkileşim: 'Bana şu gömleği ver' dersin (Konuşma). 2) Doğrulama: Satıcı cüzdanındaki parayı kontrol eder, sahte mi değil mi bakar (Kontrol). 3) Durum Değişimi: Parayı kasaya koyar, gömleği torbaya atar (Durum değişti!). Eğer satıcı sadece seninle sohbet edip gömleğin rengini övseydi ama ne paraya baksaydı ne de gömleği verseydi, ortada bir ticaret olmazdı! Yazılım da böyledir: Doğrulama ve durum değişimi yoksa orada gerçek bir iş yoktur!

🇬🇧 ENGLISH (ELI5)

Buying shoes involves three distinct actions: 1) Interaction: You ask the clerk for size 42. 2) Validation: The clerk inspects inventory to verify stock and checks your credit card for sufficient funds. 3) State Change: The card is charged, and the shoes are registered as sold. If the interaction consisted purely of browsing shoe colors without inventory verification or payment capture, no commercial transaction occurred. Software without validations and state changes is merely idle screen chatter.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Üç Adımlı Testin Mühendislik ve AI Açısından Önemi: Cockburn'ün en derin keşfi şudur: 'Kullanım senaryoları klavye başındaki ekran akışları değildir; paydaşlar arasındaki kontratın davranışsal koruyucusudur.' Sistem, klavye başında oturmayan paydaşların da çıkarlarını korumak zorundadır (örneğin şirketin parasını kaptırmaması, denetçinin log istemesi, veritabanının tutarlı kalması). Bir use case sadece Etkileşim adımlarından ('Kullanıcı butona tıklar, sistem ekranı açar') ibaretse, geliştiriciye ve AI'a hiçbir iş mantığı verilmemiş demektir! Doğrulamalar (Validation) yazılmazsa AI hiçbir güvenlik kontrolü ve kural motoru üretmez. Cockburn'ün vaka analizi: Bir şirket paydaş çıkarlarını ancak canlıya çıktıktan sonra listelediğinde, bu listenin ilk yıl gelen tüm 'Hata ve Değişiklik Talepleri' ile birebir aynı olduğu görüldü. AI çağında bu 1 yıllık gecikme ilk öğleden sonraya sıkışır: Yazılmayan her doğrulama, üretilmeyen bir güvenlik açığıdır!

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Cockburn's Three-Kind Step Axiom: Every sentence in a scenario must classify into one of three structural types: 1) Interaction between two actors (transmitting intent or payloads). 2) Validation protecting a stakeholder's interest ('System validates that member has no overdue loans'). 3) Internal State Change ('System records loan and decrements inventory'). The three-kind test is the supreme defense against pseudo-specifications that disguise UI navigation as functional requirements. When a spec omits validations and state mutations, an AI agent generates hollow presentation mockups devoid of transactional integrity. Cockburn demonstrated that omitted stakeholder interests accurately forecast a system's entire first-year change request backlog. In the AI era, this backlog manifests instantly on day one.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Üç Adımlı Testin Mimarideki Birebir Karşılığı
package com.maang.aiup.stepkinds;

public class ThreeStepTestService {
    // Domain State
    public record BookCopy(String id, boolean isAvailable) {}
    public record LoanRecord(String loanId, String memberId, String copyId, java.time.LocalDate
dueDate) {}

    public LoanRecord executeBorrowProcess(String memberId, BookCopy copy) {
        // TÜR 1: Etkileşim (Girdi parametreleri olarak geldi: memberId ve copy)

        // TÜR 2: DOĞRULAMA (Validation - Stakeholder Interest)
        if (!copy.isAvailable()) {
            throw new IllegalStateException("BR-004: Kitap kopyası ödünç verilebilir durumda değil!");
        }

        // TÜR 3: DURUM DEĞİŞİMİ (Internal State Change)
        var updatedCopy = new BookCopy(copy.id(), false); // Stok azaldı
        var loan = new LoanRecord("L-99", memberId, updatedCopy.id(),
java.time.LocalDate.now().plusDays(28));

        System.out.println("✅ 3 Adımlı Test Tamamlandı: " + loan);
        return loan;
    }

    void main() {
        var service = new ThreeStepTestService();
        service.executeBorrowProcess("MEM-1", new BookCopy("CPY-1", true));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir use case adımı 'Kullanıcı Kaydet butonuna basar' dedikten hemen sonra 'Sistem kullanıcıya ana sayfayı gösterir' diyorsa DURUN! Aradaki doğrulama ve veri tabanına kayıt (durum değişimi) adımları unutulmuştur. AI bu adımları görmezse hiçbir şey kaydetmeyen bomboş bir sayfa üretir!

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

If a step states 'User clicks Submit' followed immediately by 'System displays Success screen', HALT! The validation step and the state mutation step were completely omitted. The AI will generate a hollow screen transition that persists nothing.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Her senaryo adımı 3 türden biri olmalıdır: Etkileşim, Doğrulama veya Durum Değişimi.
- Sadece etkileşim içeren senaryolar use case değildir; ekran gezintisidir ve iş kurallarını kaybeder.
- Yazılmayan her doğrulama adımı, yapay zekanın üretmeyi unuttuğu bir güvenlik zafiyetidir.

🎯 Key Architectural Takeaways

- Every step must strictly be an Interaction, a Validation, or an Internal State Change.
- Specs containing only interactions are UI walkthroughs that forfeit all business rules.
- Every unstated validation step becomes a vulnerability omitted by the AI generator.

15

BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Hedef İrtifası: Neden Deniz Seviyesinde (Sea Level) Yazıyoruz? (Bulut ve Balık Tuzakları)

Goal Altitude: Why We Write at Sea Level (Avoiding Kite & Fish Traps)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                ALISTAIR COCKBURN'ÜN HEDEF İRTİFALARI

☁️ BULUT SEVİYESİ (KITE / CLOUD - Çok Yüksek):
"Kütüphaneyi İşlet" (Run the Library)
❌ Çok geniş! Günler/haftalar sürer. AI tahmin edemez, devasa bir monolit uydurur!
   (Bu seviye Inception aşamasındaki BPMN İş Sürecine aittir!)

🌊 DENİZ SEVİYESİ (SEA LEVEL / USER-GOAL - TAM İDEAL NOKTA!):
"Kitap Ödünç Al" (Borrow Book) / "Rezervasyon Yap" (Reserve Book)
✅ TEK KİŞİ, TEK YERDE, TEK OTURUMDA (2 - 20 DAKİKA) tamamlar!
✅ Aktör işi bitince sistemin başından mutlu ayrılır!

🐟 BALIK SEVİYESİ (FISH / SUBFUNCTION - Çok Alçak / Su Altı):
"ISBN Kodunu Doğrula" / "Açılır Menüyü Aç"
❌ Çok küçük! Kendi başına bir iş değeri yoktur. Bir use case değil, bir alt adımdır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restorana gittiğini hayal et: 1) Bulut Seviyesi: 'Beni doyur ve ömrümün sonuna kadar besle' demek gibidir (Çok genel, garson ne yapacağını şaşırır). 2) Balık Seviyesi: 'Garsona çatalı uzatmasını söyle, çatalı sol eline al, ağzını 3 santim aç' demektir (Gereksiz detayda boğulursun). 3) Deniz Seviyesi: 'Bir porsiyon mantı getir' demektir! Siparişini verirsin, yersin, hesabı öder mutlu bir şekilde masadan kalkarsın (20 dakika, tek oturum!). Use case'ler daima deniz seviyesinde yazılır!

🇬🇧 ENGLISH (ELI5)

At a restaurant: 1) Cloud Level: Demanding 'Nourish my biological existence for the next forty years.' (Absurdly broad, paralyzes the waiter). 2) Fish Level: 'Extend index finger, wrap around fork tine, elevate elbow 15 degrees.' (Absurdly granular, micromanages physical mechanics). 3) Sea Level: 'Order the ribeye steak.' You order, dine, settle the bill, and leave satisfied in 45 minutes. Use cases exist strictly at sea level.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Hedef Seviyeleri (Goal Levels) ve Mühendislik Dengesi: Cockburn use case yazımındaki en zor zanaatin doğru irtifayı bulmak olduğunu söyler: 1) Su Altı / Balık Seviyesi (Subfunctions): 'Kullanıcı Adı Kontrolü' veya 'Kredi Kartı Format Doğrulama' birer use case değildir; bir ana hedefin içindeki basit birer adımdır. Eğer bunları use case yaparsanız, sistem 500 tane anlamsız, bağlamsız, başı sonu olmayan dosya parçasına bölünür (Fragment Monolith). AI bunlardan kod üretirken bağımlılıkları bağlayamaz. 2) Bulut Seviyesi (Summary Goals): 'Hastaneyi Yönet' veya 'Tedarik Zincirini Çalıştır'. Bunlar birden fazla kişinin günlerce süren oturumlarını kapsar. Bu süreçler use case değil, BPMN iş süreçleridir. 3) Deniz Seviyesi Testi: Bir hedefin deniz seviyesinde olduğunu anlamanın iki turnusol kağıdı vardır: a) 'Neden?' sorusu bir üst seviyeyi (bulut), 'Nasıl?' sorusu bir alt seviyeyi (balık) verir. b) Tek bir aktör, tek bir yerde, tek bir oturumda (2 ila 20 dakika içinde) bitirip sistemin başından mutlu ayrılıyor mu? AIUP'ta tüm use case'ler istisnasız deniz seviyesinde yazılır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Goal Altitude Taxonomy: 1) Fish Level (Subfunction): Underwater activities like 'Validate ISBN' or 'Query Customer Record'. They deliver zero standalone business utility. Elevating them to use case artifacts fractures the architecture into hundreds of disconnected fragments that confuse LLMs. 2) Cloud Level (Summary Goal): Multi-day, multi-party business workflows ('Manage Inventory', 'Run Logistics'). These belong in Inception BPMN diagrams, not functional specs. 3) Sea Level (User-Goal Level): The sweet spot. Cockburn's litmus test: executed by one person, in one location, in one sitting (2 to 20 minutes), concluding with the actor walking away with completed business value. LLMs generate cohesive services and comprehensive test harnesses when bounded strictly to sea-level scope.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Deniz Seviyesi Use Case Sınırı (Ne Çok Büyük Ne Çok Küçük)
package com.maang.aiup.altitude;

public class SeaLevelUseCaseDemo {
    // ❌ BALIK SEVİYESİ (Ayrı bir use case DEĞİL, sadece bir metod/kuraldır):
    public static boolean validateIsbn(String isbn) {
        return isbn != null && isbn.startsWith("978");
    }

    // ✅ DENİZ SEVİYESİ (Gerçek Use Case: Tek Oturum, Tek Hedef, Tam Değer):
    public record BorrowBookRequest(String memberId, String isbn) {}
    public record BorrowBookResponse(boolean success, String receiptId) {}

    public static class BorrowBookService {
        public BorrowBookResponse execute(BorrowBookRequest request) {
            // Balık seviyesi kontroller içeride çağrılır:
            if (!validateIsbn(request.isbn())) {
                throw new IllegalArgumentException("Geçersiz ISBN!");
            }
            // Deniz seviyesi iş tamamlanır:
            return new BorrowBookResponse(true, "RCPT-" + System.currentTimeMillis());
        }
    }

    void main() {
        var service = new BorrowBookService();
        var resp = service.execute(new BorrowBookRequest("M-1", "978-0134685991"));
        System.out.println("Deniz seviyesi use case tamamlandı: " + resp);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

'Müşteri Ara' bir use case değildir! Müşteri ara butonuna basıp ekranı kapatıp evine giden kimse mutlu olmaz. Müşteri 'Siparişi İptal Etmek' için aranır; asıl use case sipariş iptalidir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

'Search Customer' is rarely a sea-level use case. Nobody logs into a banking app simply to gaze at a customer profile and log out satisfied. They search in order to transfer funds or update an address. The user goal is the transfer.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Use case'ler daima deniz seviyesinde yazılır: Tek kişi, tek yerde, tek oturumda (2-20 dk).
- Balık seviyesi hedefler (Validate ISBN) use case değil, bir use case'in içindeki adımlardır.
- Bulut seviyesi hedefler (Run Company) use case değil, Inception aşamasındaki BPMN süreçleridir.

🎯 Key Architectural Takeaways

- Use cases must be drafted at sea level: one person, in one place, in one sitting (2-20 min).
- Fish-level tasks are implementation steps or validations, not independent use cases.
- Cloud-level workflows belong in high-level BPMN business process diagrams.

16

BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Kapsam Denetimi & In/Out Listesi: Kapsam Kaymasını ve AI Halüsinasyonunu Engelleme

Scope Control & The In/Out List: Preventing Scope Creep & AI Hallucinations

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              COCKBURN'ÜN IN / OUT LİSTESİ İLE KAPSAM KİLİTLEME │
└──────────────────────────────────────────────────────────────┘

SİSTEM: KÜTÜPHANE YÖNETİM SİSTEMİ (LIBRARY SYSTEM)

KAPSAM İÇİNDE (IN)                    | KAPSAM DIŞINDA (OUT) - ASLA ÜRETME!
--------------------------------------|-----------------------------------
- Katalogda kitap arama               | - Çevrimiçi kredi kartı ile ödeme
- Kitap ödünç alma & iade             | - Sesli kitap (Audiobook) streaming
- Gecikme cezası hesaplama            | - Kargo ile eve kitap teslimatı
- E-posta ile iade hatırlatması       | - Kafeterya entegrasyonu ve kahve siparişi

🛡️ GÖREVİ: AI ajanının 'Out' kolonundaki özellikleri kafasına göre
          koda eklemesini (Scope Hallucination) %100 engeller!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir marangoza 'Bana bir çalışma masası yap' dediğinde, marangoz kendi kendine gaza gelip masanın altına mini buzdolabı, üstüne disko ışığı ve kenarına masaj aleti takmaya kalkabilir. Cockburn'ün In/Out listesi masanın üstüne koyduğun kırmızı listedir: 'Masa yapılacak: EVET. Çekmece olacak: EVET. Buzdolabı: KESİNLİKLE HAYIR! Disko topu: HAYIR!' Marangoz (veya AI) bu kırmızı listeyi görünce haddini bilir, sadece senden isteneni yapar!

🇬🇧 ENGLISH (ELI5)

Commissioning a carpenter to build a desk without boundary limits might inspire them to mount a mini-fridge, cup warmer, and neon strip lights to the wood. An In/Out list is the contract pinned to their workbench: 'Oak tabletop: IN. Two pencil drawers: IN. Refrigeration unit: OUT. Ambient LED lighting: OUT.' AI agents possess expansive latent knowledge; without an explicit OUT boundary, they aggressively generate out-of-scope fluff.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Tasarım Kapsamı (Design Scope) ve In/Out Disiplini: Cockburn'ün geliştirdiği 'In/Out Listesi', yazılım tarihindeki en ucuz ve en etkili kapsam yönetimi aracıdır. Masaya oturup 10 dakika içinde neyin sistemin içinde neyin kesinlikle dışında olduğunu bir tabloya yazmak, haftalar süren kapsam kaymasını (Scope Creep) önler. AI Unified Process'te bu liste iki kat daha hayatidir: LLM'ler eğitim verilerindeki kalıpları tekrarlamaya meyillidir. Örneğin bir 'Kütüphane Ödünç Alma' sistemi tasarlarken, AI internette gördüğü Amazon veya e-ticaret sitelerinden esinlenerek 'Kredi Kartıyla Ödeme' veya 'Ev Adresine Kargo Takibi' gibi devasa modülleri koda eklemeye kalkar. Şartnamenin ve vizyon dökümanının (`docs/vision.md`) başındaki In/Out tablosu, AI ajanının bu gereksiz özellikleri üretmesini kesin olarak bloke eder. Sistem bir kara kutu (black box) olarak sınırlandırılır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Architectural Scope Boundaries via the In/Out List: Design scope defines the black-box perimeter of the target system. Cockburn's In/Out list remains the most cost-effective governance instrument in engineering: spending 10 minutes debating boundary edge cases prevents months of scope creep. In the AI Unified Process, the In/Out list acts as a hard negative-constraint prompt for autonomous coders. LLMs inherently suffer from feature inflation: prompt an agent to build an inventory tool, and it instinctively hallucinates Stripe checkout flows or third-party courier dispatchers because modern public repos frequently couple them. Explicitly documenting the OUT column creates a structural barrier against generative overreach.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Kapsam Dışına Çıkmayı Engelleyen Sınırlandırılmış Servis Modeli
package com.maang.aiup.scope;

public class BoundedLibraryService {
    public record BorrowRequest(String memberId, String bookId) {}

    // IN Kapsamı: Sadece kütüphane içi ödünç alma işlemi
    public void processBorrow(BorrowRequest req) {
        System.out.println("IN Kapsamı: Kitap ödünç işlemi yapıldı: " + req.bookId());
    }

    // OUT Kapsamı Uyarısı: AI bu metodları uydurmaya kalkarsa derleme/mimari testleri engeller
    // public void processOnlineCreditCardPayment(...) -> YASAK (docs/vision.md OUT listesi!)
    // public void dispatchHomeDeliveryCargo(...)      -> YASAK (docs/vision.md OUT listesi!)

    void main() {
        var service = new BoundedLibraryService();
        service.processBorrow(new BorrowRequest("M-10", "BK-99"));
        System.out.println("✅ Kapsam sınırları başarıyla korundu.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

In/Out listesi yaparken OUT kolonunu boş bırakmayın! Bir sistemde neyin yapılmayacağını söylemek, neyin yapılacağını söylemek kadar önemlidir. AI 'yapılmayacaklar' listesinden güç alır.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never leave the OUT column empty. Declaring what the system will NOT do is just as critical as defining its features. Negative constraints narrow the AI's generation space exponentially.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- In/Out listesi 10 dakikada hazırlanır ve haftalar sürecek kapsam kaymasını engeller.
- AI geniş eğitim verisi yüzünden kapsam dışı e-ticaret/ödeme özellikleri uydurmaya meyillidir.
- OUT kolonu, yapay zekanın tasarım uzayını daraltan en güçlü negatif kısıttır.

🎯 Key Architectural Takeaways

- An In/Out list takes 10 minutes to draft and prevents weeks of costly scope drift.
- AI is prone to hallucinating out-of-scope enterprise features from its training data.
- The OUT column provides the strongest negative prompt boundary for code generation.

17

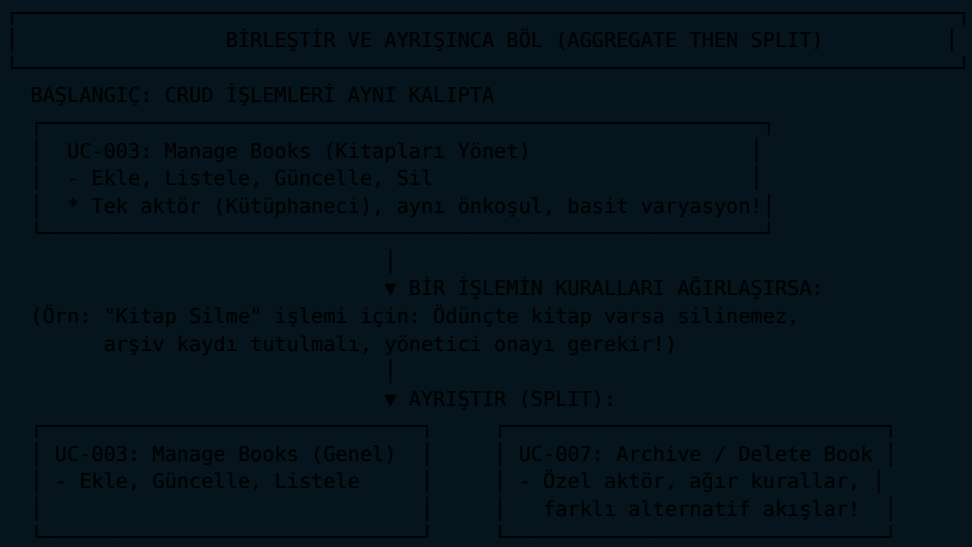
BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Önce Birleştir, Akışlar Ayrışınca Böl: 'Manage X' Kuralı ve CRUD Birleştirmesi

Aggregate First, Split When Flows Diverge: The 'Manage X' Rule

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda 'İçecekler' menüsü vardır; çay, kahve, su ve kola hepsi aynı sayfadadır çünkü hepsi bardağa dökülüp servis edilir (Basit CRUD). Ama müşteri 'Özel İsviçre Sıcak Çikolatası' istediğinde, aşçı özel tencere çıkarır, 15 dakika çikolata eritir, içine marshmallow koyar. İşte o zaman o içecek genel menüden çıkarılıp 'Özel Tatlılar' başlığına taşınır! Başta her şeyi bir sepete koy, kuralları karmaşıklaşan şeyi kendi özel kutusuna ayır!

🇬🇧 ENGLISH (ELI5)

On a café menu, standard drinks (tea, drip coffee, water) share a single section because they require identical preparation steps. But when the café introduces traditional siphon-brewed pour-overs involving specialized burners, timing scales, and origin certifications, that beverage earns its own dedicated specialty menu card. Start standard CRUD operations in one consolidated container; split them only when operational rules diverge.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Use Case Sayısı Optimizasyonu ve 'Manage X' Kuralı: Yeni başlayan mimarların en büyük hatası, 4 tabloluk bir veritabanı için 25 tane use case yazmaktır (Create Book, Read Book, Update Book, Delete Book, Search Book…). Bu durum şartname enflasyonuna yol açar ve AIUP'un 'küçük ve okunabilir model' felsefesini yok eder. Cockburn ve Martinelli'nin altın kuralı: 'Aggregate first, split when the flows diverge' (Önce birleştir, akışlar ayrışınca böl). Bir varlığın CRUD işlemleri aynı aktör (Kütüphaneci) tarafından yapılıyor ve temel bir başarı akışını paylaşıyorsa, başlangıçta tek bir 'UC-003: Manage Books' use case'i olarak yazılır. Ancak bu işlemlerden biri (örneğin Block Customer veya Archive Book) kendine has ağır iş kuralları, farklı önkoşullar veya farklı alternatif akışlar geliştirmeye başladığı an, o operasyon derhal 'UC-008: Block Customer' olarak bağımsız bir use case'e dönüştürülür. Bu sayede küçük bir mikroservis 40 değil, sadece 4 ila 8 use case ile kusursuz şekilde modellenir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Managing Use Case Cardinality via Aggregate-then-Split: Novice analysts routinely generate twenty distinct use cases for basic entity management (Create User, Edit User, View User, Delete User, Search User), creating specification bloat that overwhelms engineering reviews. The Martinelli-Cockburn heuristic dictates: start every CRUD resource as a single consolidated 'Manage X' use case. The aggregate holds cleanly as long as the operations share a single primary actor, identical preconditions, and standard data persistence steps. However, the precise moment an operation diverges—e.g. 'Block Account' requiring fraud checks, compliance event logging, and secondary approval flows—it must be excised into an action-level use case. This discipline bounds standard microservices to 4 to 8 use cases total.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 'Manage X' Çoklu İşlem Yöneticisinin Ayrışma Noktası
package com.maang.aiup.aggregate;

public class ManageBooksService {
    public record Book(String id, String title, String author, boolean isArchived) {}

    // Standart CRUD: Tek bir servis/use case içinde birleşik yaşar
    public void createBook(Book b) { System.out.println("Kitap eklendi: " + b.title()); }
    public void updateBook(Book b) { System.out.println("Kitap güncellendi: " + b.title()); }

    // AYRIŞMA NOKTASI (Ayrı bir Use Case adayı: ArchiveBookUseCase):
    // Ağır iş kuralları: Aktif ödünç kontrolü, denetim kaydı, bildirim!
    public void archiveBook(String bookId, int activeLoanCount) {
        if (activeLoanCount > 0) {
            throw new IllegalStateException("BR-030: Ödünçte kopyası bulunan kitap arşivlenemez!");
        }
        System.out.println("Kitap arşivlendi ve denetim logu üretildi: " + bookId);
    }

    void main() {
        var service = new ManageBooksService();
        service.createBook(new Book("B1", "Clean Code", "Robert Martin", false));
        service.archiveBook("B1", 0);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir aktör sisteme asla 'Müşterileri yönetmek' amacıyla oturmaz. Birleştirilmiş 'Manage Customers' use case'i sadece benzer CRUD operasyonlarını bir arada tutan bir başlangıç konteyneridir; kurallar ağırlaştığında bölmekten çekinmeyin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

An end-user never logs in with the generic goal to 'manage customers'; they come to add or block someone. The 'Manage X' pattern is a structural convenience for uniform CRUD. Split it immediately when business logic diverges.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Başlangıçta CRUD işlemlerini 'Manage X' altında tek bir use case olarak toplayın.
- Bir operasyon kendi özel iş kurallarını ve hata akışlarını geliştirdiğinde hemen ayrı bir use case'e bölün.
- Bu kural sayesinde tipik bir mikroservis 40 dağınık use case yerine 4-8 güçlü use case ile modellenir.

🎯 Key Architectural Takeaways

- Start CRUD operations consolidated under a single 'Manage X' use case.
- Split out an action-level use case the moment preconditions or business rules diverge.
- This aggregation keeps a microservice clean at 4 to 8 comprehensive use cases instead of forty.

18

BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Kullanıcının Diliyle İsimlendirme: Neden 'Kitap Ödünç Al', 'Ödünç Kaydı Oluştur'dan İyidir?

Naming in the User's Language: Why 'Borrow Book' Beats 'Create Loan Record'

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                İSİMLENDİRME FELSEFESİ: KULLANICI DİLİ VS TEKNİK JARGON

❌ GELİŞTİRİCİ DİLİ (TEKNİK / VERİTABANI ODAKLI):
- "Create Loan Record" (Ödünç Kaydı Oluştur)
- "Insert User Entity" (Kullanıcı Varlığı Ekle)
- "Update Account Balance Table" (Bakiye Tablosunu Güncelle)
💥 Hata: Paydaşın kafasını karıştırır, sistemi veritabanı CRUD operasyonuna indirger!

✅ KULLANICI DİLİ (İŞ HEDEFİ ODAKLI):
- "Borrow Book" (Kitap Ödünç Al)
- "Register Member" (Üye Kaydet)
- "Transfer Funds" (Para Transfer Et)
🎯 Başarı: Aktif fiil öbeğidir, gerçek dünya hedefini anlatır, her 3 okuyucu da anlar!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kütüphaneye gittiğinde görevliye 'Affedersiniz, veritabanınızdaki ödünç tablosuna yeni bir satır ekleyip benim ID'mi o satırla ilişkilendirmek istiyorum' demezsin! Görevliye sadece 'Şu kitabı ödünç almak istiyorum' dersin. Bir use case'in adı, kullanıcının ağzından çıkan o doğal cümle olmalıdır. Teknik kelimeleri arkadaki motora bırak; tabelaya herkesin anladığı ismi yaz!

🇬🇧 ENGLISH (ELI5)

When visiting a library, you never announce to the desk clerk: 'Greetings, I wish to initiate an INSERT transaction on your LOANS table linking my foreign key to this asset ID.' You say: 'I would like to borrow this book.' The title of a use case must always match the natural words spoken by the human pursuing the goal. Keep technical jargon inside the engine; keep the billboard clean.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İsimlendirme Semantiği ve Arayüz Olarak İsim Kuralı: Cockburn ve Martinelli use case isimlerinin yalnızca birer başlık değil, sistemin 'dış arayüzü' (interface) olduğunu vurgular: 1) Aktif Fiil Öbeği (Active Verb Phrase): İsim daima bir eylemle başlamalıdır: 'Borrow Book', 'Enroll in Course', 'Checkout Cart'. 'Kitap İşlemleri' veya 'Kullanıcı Modülü' gibi isimler use case olamaz çünkü bir hedef belirtmez. 2) Kullanıcı Dili (User Domain Language): 'Create Loan Record' ismi bir hedefin değil, veritabanındaki bir yazma operasyonunun adıdır. Eğer bir use case'i bir hedef olarak adlandıramıyorsanız, henüz elinizde bir use case yok demektir! 3) Birebir Eşleşme (Verbatim Interface): Use case diyagramındaki (PlantUML) isim ile şartname dosyasının adı (`UC-002-borrow-book.md`) ve dökümanın içindeki `# Use Case: Borrow Book` başlığı noktası virgülüne kadar aynı olmalıdır. Bu kural downstream çalışan kod üretim araçlarının (`/implement`) dökümanları hatasız bulmasını sağlar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Naming as an Architectural Interface: 1) Active Verb-Noun Syntax: Use case names must strictly form active verb phrases representing user objectives: 'Borrow Book', 'Register Account', 'Submit Expense Claim'. Generic labels like 'Book Processing' or 'User Maintenance' fail because they denote static categories rather than goal-directed behaviors. 2) User Vocabulary vs Schema Projections: 'Create Loan Record' describes an internal database mutation; 'Borrow Book' captures the real-world business transaction. If you cannot articulate the name as a tangible user goal, the requirements boundaries are premature. 3) Verbatim Traceability Interface: The label in the PlantUML diagram (`UC-002\nBorrow Book`), the filename (`UC-002-borrow-book.md`), and the title header must match identically. Downstream AI tools rely on strict lexical matching to bridge architectural models with source code generation.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Kullanıcı Dilini Doğrudan Kod Semantiğine Taşıma (Ubiquitous Language)
package com.maang.aiup.naming;

public class DomainNamingDemo {
    // ❌ KÖTÜ TEKNİK İSİMLENDİRME:
    // public class LoanRecordCreator { public void insertRow(...) {} }

    // ✅ TEMİZ VE KULLANICI ODAKLI İSİMLENDİRME (Use Case ile Birebir):
    public static class BorrowBookUseCase {
        public record Command(String memberId, String bookIsbn) {}
        public record Result(String loanId, String dueDate) {}

        public Result borrowBook(Command cmd) {
            // Kod şartnamenin dilini konuşur!
            System.out.printf("Kitap ödünç alındı: Üye %s -> Kitap %s%n", cmd.memberId(),
cmd.bookIsbn());
            return new Result("L-101", "2026-11-17");
        }
    }

    void main() {
        var uc = new BorrowBookUseCase();
        uc.borrowBook(new BorrowBookUseCase.Command("MEM-9", "ISBN-456"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Use case adında 'Veritabanı', 'Kayıt', 'Tablo', 'API', 'Servis' gibi yazılımcı kelimeleri geçiyorsa o isim yanlıştır. Hemen o kelimeleri silin ve kullanıcının gerçek hayattaki eylemini yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

If words like 'Database', 'Record', 'Table', 'Entity', or 'API' appear in your use case title, the name is corrupted by implementation bias. Strip away technical nouns and state the actor's real-world goal.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Use case ismi daima kullanıcının dilinde kısa ve aktif bir fiil öbeğidir (Borrow Book).
- Veritabanı jargonu ('Create Loan Record') hedefi değil, uygulama mekanizmasını anlatır ve yanlıştır.
- Diyagramdaki isim, dosya adı ve başlık noktası virgülüne kadar birbirini tutmalıdır.

🎯 Key Architectural Takeaways

- Use case titles must be short active verb phrases in user language (e.g. Borrow Book).
- Database jargon describes technical mutations, failing to capture user goals.
- The diagram name, markdown file name, and header title must match verbatim.